GetEvents — Where the Project Stands

What we're building, how data flows through it, what's working today, and what's next.

THE VISION, IN ONE LINE

> **GetEvents is an event intelligence API** — not a listings website. Other companies' software (travel apps, hotel pricing engines, staffing tools) will query us for what's happening, where, and how big a deal it is: concerts, sports, conferences, festivals, public holidays, disasters, severe weather, health warnings, and more, pulled from hundreds of sources worldwide.

HOW ONE PIECE OF DATA FLOWS THROUGH THE SYSTEM

Every event, from every source, passes through the same pipeline before it's considered clean enough to serve to a customer:

1. **Connector** fetches raw events from one specific source. `(Ticketmaster, a scraper, a weather feed...)`
2. **Raw storage** saves it exactly as received, untouched.
3. **Normalizer** rewrites every source's format into one common shape.
4. **Deduplicator** merges repeat reports of the same event.
5. **Entity resolution** confirms the venue/artist/organizer are known entities.
6. **Geocoder** attaches exact map coordinates and timezone.
7. **Classifier** files the event under the right category.
8. **Attendance estimator** forecasts expected crowd size.
9. **Impact scoring** rates how much the event matters to its surroundings.
10. **Canonical database** holds the one clean, authoritative record.

11 **API** — where customers' software asks for and receives the data.

BUILD STRATEGY

We are deliberately starting small and proving the pipeline shape with one real source before scaling out, rather than building all 1000+ connectors up front.

Connectors as plugins

Each source (API or scraper) is a small, independent unit with the same four jobs: discover, fetch, normalize, validate. Adding source #501 later shouldn't require touching anything else.

Source registry

A hand-maintained file lists every source and which countries it covers. Asking for events in a city automatically resolves to only the sources relevant to that country.

Shared coordinates database

A lightweight local database caches venue coordinates the first time they're found, so repeated events at the same venue never need re-geocoding.

Mixed source types

1–2 real APIs (Ticketmaster and similar) plus many scraped sources, some country-specific, some multi-country — the registry treats both the same way.

CURRENT STATUS

- [x] Full system architecture designed — bounded contexts, pipeline stages, folder structure documented
- [x] Local development environment set up (isolated Python environment, dependency tracking)
- [x] Ticketmaster connector prototype built — fetches, normalizes, and validates real events end-to-end
- [x] Venue coordinates confirmed available directly from Ticketmaster — no geocoding needed for this source
- [x] Shared coordinates database built and tested (first-writer-wins on conflicts)
- [x] Source registry built and tested — correctly filters sources by country
- [ ] Wire Ticketmaster connector to write into the shared coordinates database *(immediate next step)*

ROADMAP — NEXT STEPS

1. Finish wiring Ticketmaster to the coordinates database and registry-based dispatch.
2. Onboard the other 1–2 confirmed API sources using the same pattern.
3. Build the first scraper-based connector to prove the "geocode on cache-miss" path works end-to-end.
4. Formalize working connectors into the plugin structure (standard base class, manifest file per source).
5. Stand up shared infrastructure (proper database, message queue) once several connectors are running concurrently.
6. Scale out scraper connectors country by country, prioritized by target markets.

OPEN QUESTIONS FOR DISCUSSION

Priority markets	Which countries/cities should we prioritize first once we move past France as the proving ground?
Source budget	Are paid geocoding APIs or scraping proxy services in scope if free tiers/free geocoding become a bottleneck?
Timeline	What's a realistic checkpoint for "N sources live end-to-end" to treat as the next milestone?